CS 530 Lab Notebook

Lab 1

Name: Jarren Calizo
OdinID: calizo
Date: October 5, 2025

Table of Contents

Lab Overview 2
Lab 1.2 ARP, Wireshark, Netsim 3
    Step 1: Examining ARP on linux.cs.pdx.edu 3
    Step 2: Sorting and Counting ARP Entries 5
    Step 3: ARP (Cloud) 7
    Step 4: Netsim Completion 9
Lab 1.3 Cloud Networking 13
    Step 1-3: Network Scanning and Service Discovery 13
    Step 4-5: CIDR and Default Network Exploration 15
    Step 6: Creating Custom Networks 17
    Step 7: Clean Up 20

Lab Overview

Lab 1 served as my first deep dive into how the Internet’s fundamental layers interconnect - from the physical whispers of Ethernet to the abstract vastness of cloud-based virtual networks. The lab unfolded in two major parts: exploring the Address Resolution Protocol (ARP) and packet behavior on traditional systems, and then extending those principles into Google Cloud’s virtualized environment. Together, they traced a narrative of visibility - from the unseen handshakes beneath local IP communication to the orchestrated architecture of the cloud.

In Lab 1.2 (ARP, Wireshark, Netsim), I began by examining how IP and MAC addresses form the backbone of local discovery. Using commands like ip address, arp -a, and netstat -rn, I identified my PSU host’s physical and logical identities, uncovering how ARP binds digital intent to hardware existence. Sorting and counting ARP entries revealed subtle relationships in subnet mapping, while observing the same logic on a Google Cloud VM reaffirmed that even virtualized systems rely on the same trust-based translation layer. The Netsim exercises then transitioned theory into simulation - walking through packet creation, routing, spoofing, DoS amplification, and man-in-the-middle interception. Each scenario felt like peeling back the Internet’s skin, seeing how every packet is both structured by order and vulnerable by design.

In Lab 1.3 (Cloud Networking), I shifted focus to the scalability and segmentation of modern cloud networks. Using nmap, I performed subnet-wide scans across multiple instances, revealing the pulse of open ports and active services - each a reminder that every system, however isolated, speaks in traffic. Exploring Google Cloud’s default VPC networks and custom subnet creation taught me the elegance of software-defined networking: boundaries drawn not by cables, but by intent. The moment pings failed between my default and custom networks, I grasped how isolation becomes security, and how connectivity must always be earned, not assumed.

Ultimately, Lab 1 grounded my understanding of the Internet not as an abstraction, but as a living system of trust, translation, and topology. From ARP tables on a PSU server to custom networks spanning continents, every exercise echoed a simple truth: technology, no matter how advanced, remains an orchestration of human logic - visible only when we look closely enough.

Lab 1.2 ARP, Wireshark, Netsim

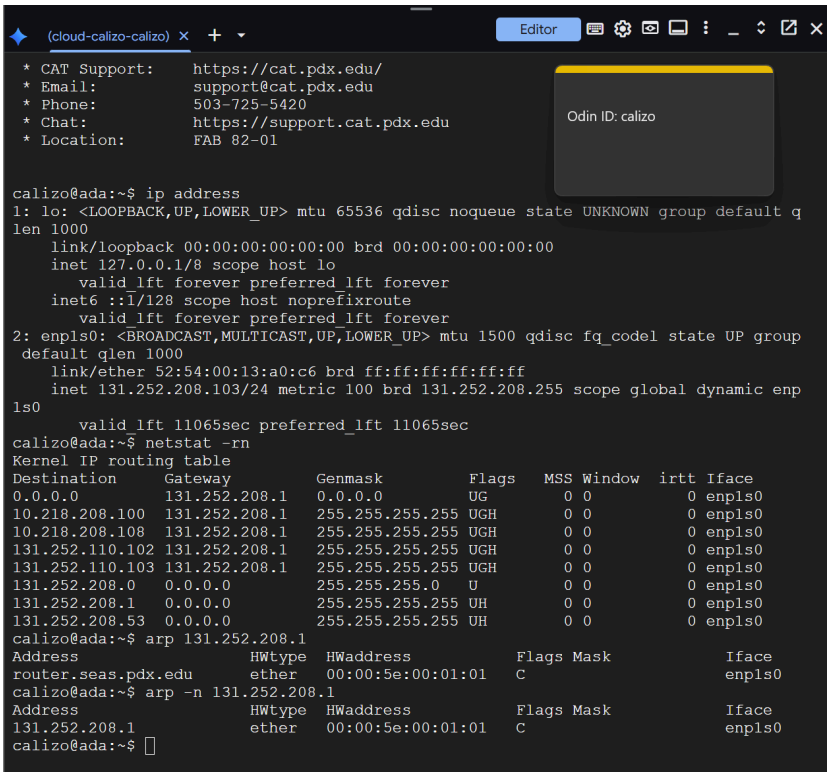
Step 1: Examining ARP on linux.cs.pdx.edu

This step explored how the Address Resolution Protocol (ARP) connects two worlds in networking: logical IP addressing and physical hardware communication. By examining ARP on a PSU Linux host, I learned how local systems recognize each other beneath the surface of the network layer.

Identifying the Local Ethernet Interface

Using ip address, I identified my active interface as enp1s0, which carried both my logical and physical network identity.

- IPv4 address: **131.252.208.103**/24
- Hardware (MAC) address: **52:54:00:13:a0:c6**



The IP address reflects my position within the PSU subnet, while the MAC address defines my unique fingerprint at the data-link layer. Together, they serve as the machine's voice in a crowded digital space.

Viewing the Routing Table

Next, I ran netstat -rn. The routing table revealed the default gateway handling external traffic beyond the local network.

- Default router (gateway): **131.252.208.1**

- Interface: enp1s0

This gateway acts as a bridge between my host and the wider Internet, forwarding packets that reach beyond the familiar 131.252.208.0/24 subnet.

Querying the Default Router via ARP
I examined how my host communicates with its gateway using

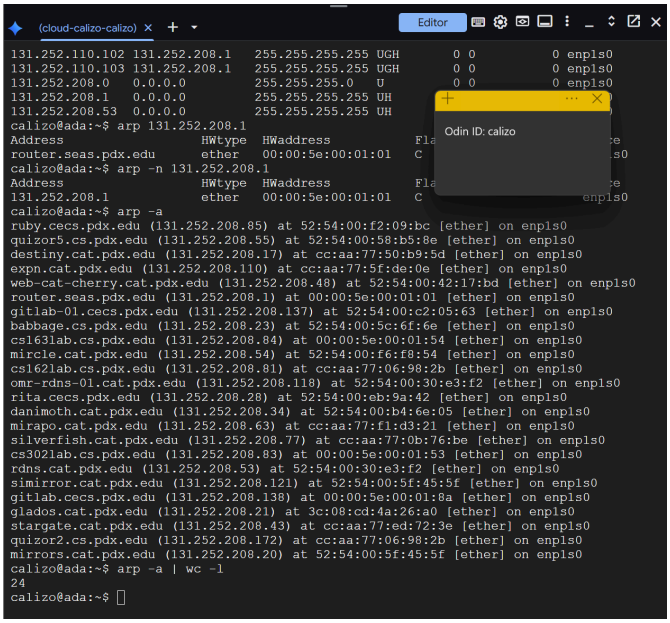
- arp 131.252.208.1
- arp -n 131.252.208.1

Results showed:

- Router name: **router.seas.pdx.edu**
- Router MAC address: **00:00:5e:00:01:01**

Using the numeric flag -n confirmed the association without DNS resolution. The 00:00:5e prefix signifies a virtualized or shared network device, fitting for an academic environment where many hosts coexist on shared infrastructure.

Listing the Entire ARP Table
To view every known peer within the subnet (arp -a) and to count them (arp -a | wc -l), there were **24 entries** in total, representing other systems and course servers actively communicating across the network, each with a unique IP-to-MAC mapping.



Insight

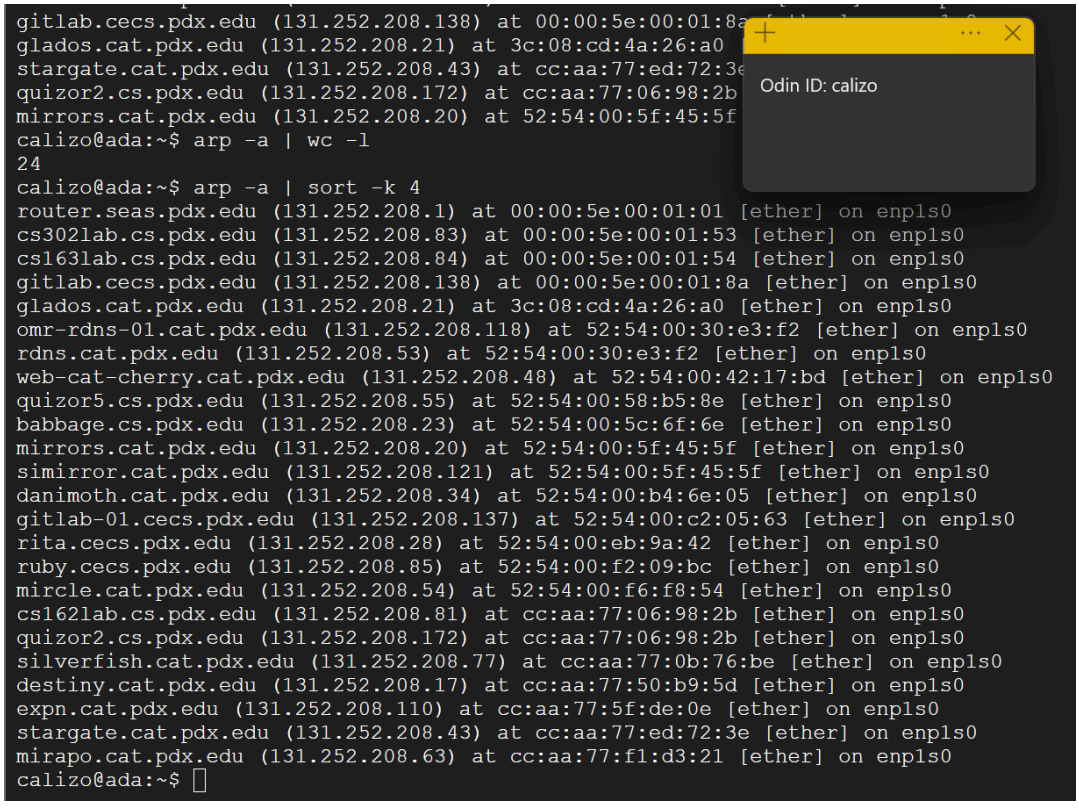
This lab made visible what is usually invisible. ARP sits quietly between layers, translating human logic into machine language. It's the unseen handshake that lets one host discover another. Seeing my local host, its neighbors, and the PSU gateway together reminded me that the Internet's vast structure is ultimately built on countless, small exchanges like these - each device trusting another enough to listen.

Step 2: Sorting and Counting ARP Entries

This step examined how multiple IP addresses can map to a single hardware address within a subnet. By sorting and parsing the ARP table, I aimed to uncover the relationships between IPs, MACs, and network topology on the PSU Linux server.

Sorting by Hardware Address

I began by identifying which field contained the MAC address in the ARP output. Using arp -a | sort -k 4, I confirmed that field 4 held the hardware address and sorted the entries accordingly. The result grouped machines by their MACs, revealing whether multiple IPs were bound to the same interface. In this case, no two distinct IPs shared an identical MAC, suggesting each machine on the network had a unique network interface.

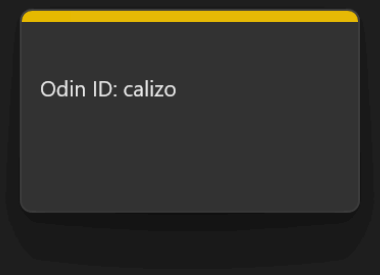


Counting Unique Hardware Addresses

To quantify uniqueness, I used the following command:

- arp -a | sort -k 4 | awk '{print $4}' | uniq | wc -l

```
calizo@ada:~$ arp -a | sort -k 4 | awk '{print $4}' | uniq | wc -l
21
calizo@ada:~$ arp -an | awk -F '[()]' '{print $2}' > arp_entries
calizo@ada:~$ ls
arp_entries  Desktop  dev_html  Documents  Downloads  public_html  smb_files
calizo@ada:~$ cat arp_entries
131.252.208.85
131.252.208.55
131.252.208.17
131.252.208.110
131.252.208.48
131.252.208.1
131.252.208.137
131.252.208.23
131.252.208.84
131.252.208.54
131.252.208.81
131.252.208.118
131.252.208.28
131.252.208.34
131.252.208.63
131.252.208.77
131.252.208.83
131.252.208.53
131.252.208.121
131.252.208.138
131.252.208.21
131.252.208.43
131.252.208.172
131.252.208.20
calizo@ada:~$
```



This returned **21 unique hardware addresses**. Since my ARP table contained **24 total IP entries**, there were **3 fewer hardware addresses** than IP addresses. This implies that three IP addresses were mapped to existing MACs, possibly due to virtual interfaces, caching behavior, or aliasing within the PSU subnet.

Generating a File of IP Addresses

To capture every IP in the ARP table, I executed:

- **arp -an | awk -F '[()]' '{print $2}' > arp_entries**

This command filtered out each IP enclosed in parentheses and wrote them into a new file called arp_entries. Verifying with cat arp_entries displayed a clean list of all 24 IP addresses on separate lines.

Network Prefix Analysis

Inspecting the arp_entries file revealed that nearly all hosts shared the same prefix:

- 131.252.208.x

This indicates that most systems reside within the **131.252.208.0/24** subnet, the primary local network used by PSU's Computer Science department. It demonstrates how devices under a shared domain are interconnected at the data-link level through a common broadcast boundary.

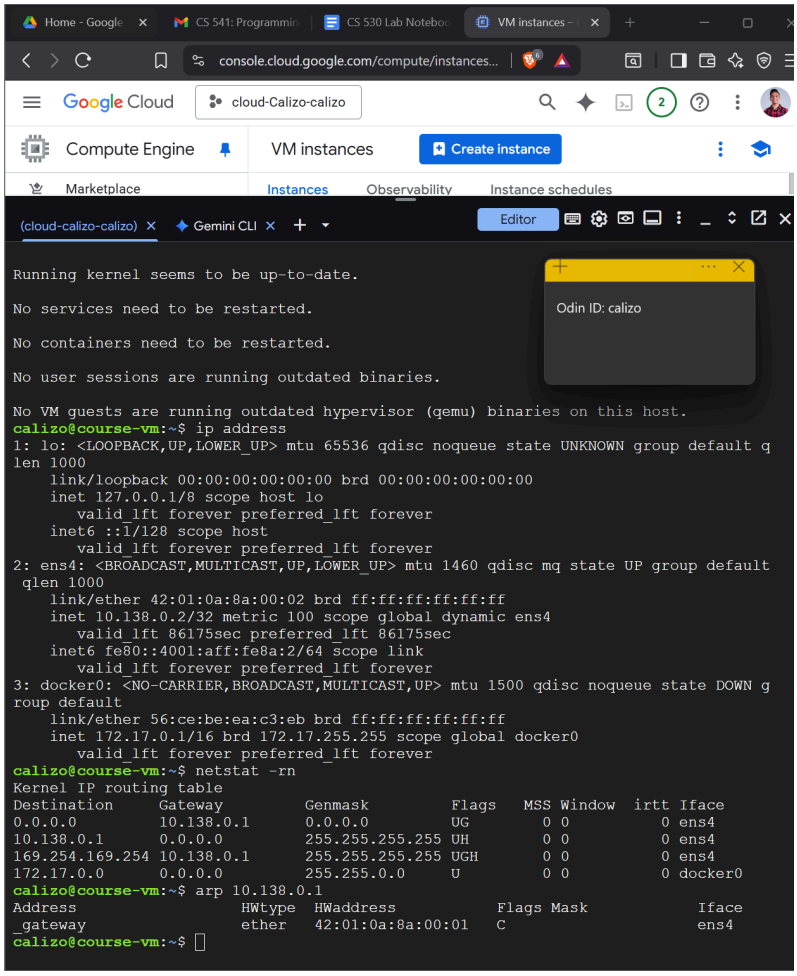
Step 3: ARP (Cloud)

This step extended the ARP exploration into a cloud environment. The goal was to observe how IP-to-MAC relationships manifest in a virtualized network, where cloud infrastructure abstracts much of the underlying hardware. By examining my Google Cloud Ubuntu VM, I could see how these mappings persist even in environments that feel far removed from physical machines.

Local Interface, IP, and Hardware Address

After starting my VM and updating its packages, I ran:

- ip address



From the output, the active interface was ens4, assigned to the VM's virtual network card.

- IPv4 address: **10.138.0.2**/32
- Hardware (MAC) address: **42:01:0a:8a:00:02**

Even within a cloud instance, this MAC address uniquely identifies the virtual adapter. The IP reflects my VM's logical location within Google Cloud's private network, reminding me that abstraction doesn't erase identity - it simply reshapes how we see it.

Default Router IP Address
I examined the routing table with:

- netstat -rn

The default route pointed to:

- Default router (gateway) IP: **10.138.0.1**
- Interface: ens4

This gateway is Google Cloud's virtual router, quietly handling all outbound traffic from my VM to the wider network. It functions as a shared link between multiple instances, yet each connection still feels personal - a small reflection of the Internet’s collective architecture.

Default Router Hardware Address
To resolve the router's hardware address, I used:

- arp 10.138.0.1

The output revealed:

- Router hardware address: **42:01:0a:8a:00:01**

This address mirrors my own interface’s pattern, suggesting both belong to the same virtualized subnet space managed by the cloud infrastructure. Even at this abstract layer, the ARP relationship still anchors every connection to something tangible - a link between identifiers that ensures trust and reachability.

<u>Insight</u>
What stands out in this step is how the cloud preserves the same networking fundamentals that govern physical systems. The ARP table, gateway, and MAC mapping all remain - only their environment has shifted. It’s a quiet reminder that

even in virtual spaces, the Internet’s structure is still built on the same human-designed rules of communication. The addresses may be synthetic, but the logic - the handshake between machines - remains deeply real.

Step 4: Netsim Completion

I completed every level in the Netsim simulator and captured the final completion screen (fireworks!). The exercise walked through basic packet mechanics, spoofing, DoS techniques, MitM, censorship evasion, and traceroute. Each level reinforced a small, concrete networking principle - how packets are formed, how devices learn and forward, and how protocol behaviors can be abused or leveraged. Below are the per-level notes, what I sent, and why it worked.

Basics

1. Getting started
    a. Actions
        i. Restarted and paused the simulator, inspected devices and packets, added a blank packet with the + button, and sent it.
    b. Why it matters
        i. Familiarized me with the UI and the packet inspector so I could read headers and follow packet flows.

2. Packet fields
    a. Actions
        i. Created a packet from Alice to Bob, set srcip and dstip in the network layer to match observed values, added the packet, then sent it.
    b. Why it matters
        i. Reinforces that packets carry source and destination information at the network layer - the router forwards on dstip, not on the "From" UI field.

3. Ping
    a. Actions
        i. Sent four additional ICMP echo requests from Alice to Google. For each packet I set proto = ICMP (capitalized) under transport.
    b. Why it works
        i. ICMP echo request/reply is a simple transport-level handshake - ping elicits immediate reply if the host is reachable.

4. Routing

a. Actions
    i. Sent a packet from Bob to Carol (srcip = Bob, dstip = Carol).
b. Why it matters
    i. Demonstrates routers use routing tables to forward packets based on dstip; end hosts do not need route knowledge.

5. Modems
    a. Actions
        i. Sent an ICMP ping from Alice to Google. Observed the modem set a single outward address for many internal hosts (source rewriting).
    b. Why it matters
        i. Shows NAT/modem behavior - multiple internal hosts can share one external address while mappings preserve return-path semantics.

Spoofs

6. IP Spoofing
    a. Actions
        i. From Alice (sending host) created a packet with srcip = Carol (spoofed), dstip = Bob. Sent the spoofed packet.
    b. Why it works
        i. The network trusts the packet header as-is. Without source authentication, receivers assume the srcip is genuine - a simple spoof.

7. Stealing Packets (switch learning exploit)
    a. Actions/Timing
        i. Timed a packet injection so it arrived when the switch was learning. Sent a packet claiming srcip = Charlie (or used the same source MAC semantics); the switch updated its MAC table to map Charlie to my port. Then packets destined for Charlie were forwarded to Alice (me).
    b. Why it works
        i. Switches learn source-MAC-to-port mappings reactively. If you inject a frame that convinces the switch that a victim MAC is on your port at the critical moment, subsequent frames get forwarded to you. Timing is the exploit.

Denial of Service

8. Basic DoS
    a. Actions
        i. Sent one packet configured and used the Repeat function for 10,000 repeats targeting Google.
    b. Why it works
        i. Overwhelms the target with more packets than it can process - fills buffers or saturates link/bandwidth and causes service failure.

9. Distributed DoS
    a. Actions
        i. Used Alice + Zombie 1 + Zombie 2 + Zombie 3 to send packets to Google, each set to repeat 100 times.
    b. Why it works
        i. With multiple sources, a single-source block or firewall is ineffective. The aggregate traffic from several senders exceeds capacity - classic DDoS.

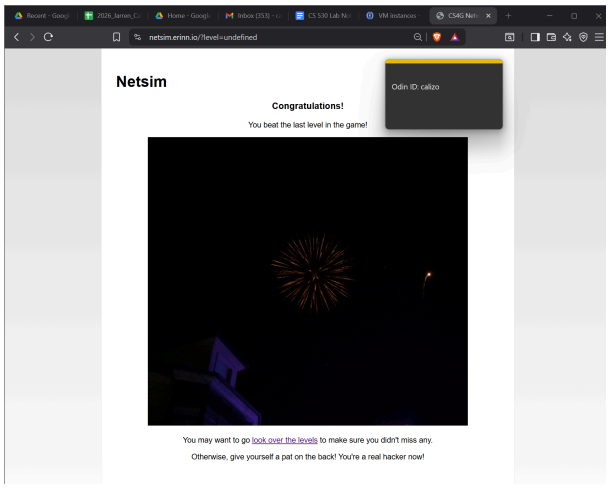
10. Smurf attack
    a. Actions
        i. From Alice, set srcip = Google (spoofed), dstip = Broadcast, proto = ICMP, Repeat = 10,000.
    b. Why it works
        i. Broadcast ping causes every host on the broadcast domain to echo the ping - they all reply to the spoofed source (Google). One crafted packet produces many replies, amplifying traffic massively and flooding the victim.

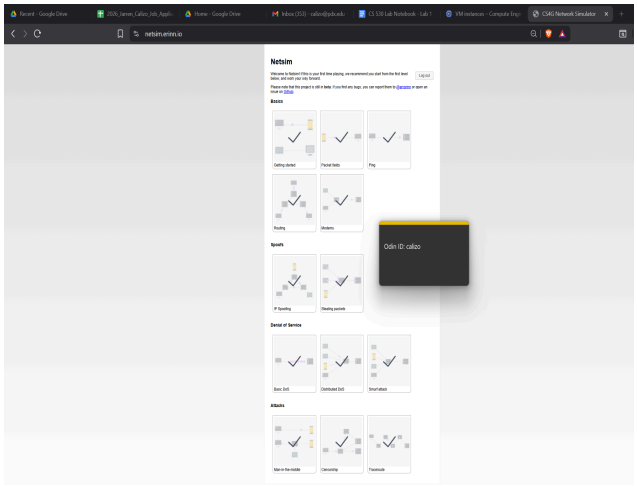
Attacks

11. Man-in-the-middle
    a. Packets I used
        i. Packet 1: From Eve, srcip = Alice, dstip = Bob, proto = encryption, type = keyrequest
        ii. Packet 2: From Eve, srcip = Bob, dstip = Alice, proto = encryption, type = keyresponse, key = 31337
        iii. Packet 3: From Eve, srcip = Alice, dstip = Bob, proto = encryption, type = message, key = 123456
    b. Why it works
        i. The key-exchange protocol lacks authentication. Eve injects a spoofed keyresponse so Alice adopts Eve's key (31337). Eve

decrypts Alice's ciphertext, re-encrypts it with Bob's genuine key, and forwards it - both endpoints believe they are talking securely to each other while Eve reads everything. The root cause is trust without cryptographic authentication.

12. Censorship
    a. Packet: From Alice, srcip = Alice, dstip = Proxy.
    b. Why it works
        i. The censor only sees Alice-to-proxy traffic. The proxy forwards to the blocked site on Alice's behalf, so the censor cannot distinguish allowed proxy traffic from illicit proxying. Using a proxy tunnels the blocked request through an allowed connection.
13. Traceroute
    a. Actions / probes
        i. Probed Google with ICMP using TTL = 1, 2, 3, 4 to elicit "Time Exceeded" replies - revealed the 4 hidden routers.
        ii. After discovering IPs (Waterloo, Toronto, New York, Mountain View in the simulator), I sent direct pings to each router to confirm reachability.
    b. Why it works
        i. Routers decrement TTL and send back ICMP time-exceeded when TTL reaches zero. Traceroute leverages this by increasing TTL stepwise so each intermediate router returns its identity. Once discovered, routers can be pinged directly.





Insight

This step helped me connect how data moves between physical and logical layers of a network and why protocols like ARP and DHCP are critical in ensuring that process

runs smoothly. Through the slides, I learned that ARP acts as a bridge between MAC and IP addressing - translating logical IPs into hardware MAC addresses so packets can reach the correct devices within a local network. Its lack of authentication, however, exposes vulnerabilities like ARP spoofing and man-in-the-middle attacks, reinforcing the importance of link-layer security. DHCP expanded my understanding of how hosts dynamically acquire IP configurations, including their default router and DNS settings, automating what would otherwise be a manual network setup.

Transitioning into IP and routing, I saw how routers use ARP and ICMP messages (like TTL expiration and traceroute responses) to forward packets across multiple networks. This highlighted how each protocol layer depends on the one beneath it - ARP at the data link layer, IP at the network layer, and ICMP providing diagnostic communication. Overall, these slides deepened my grasp of how interconnected and fragile network operations are, where even small misconfigurations or spoofed responses can have major security and routing implications.

Lab 1.3 Cloud Networking

Step 1-3: Network Scanning and Service Discovery

Step 1 - Installing nmap on the Cloud VM
To begin the lab, I started my course-vm instance and connected through SSH using the Google Cloud CLI. After verifying the system was up to date, I installed nmap with:

- sudo apt update
- sudo apt install nmap -y

A quick version check confirmed a successful installation. At that point, my VM was fully equipped to perform internal subnet scans.

Step 2 - Launching Target Instances
Next, I created three Compute Engine VMs from pdx-cs custom images within zone us-west1-b:

- lab-target-1 - introsec-enterprise-server
- lab-target-2 - kali-rolling-latest
- lab-target-3 - metactf-u20-2025-08-25

Each VM was configured to allow HTTP traffic only and to remain on the same internal network for scanning. Once deployed, they appeared in my Compute Engine console

with the internal IPs 10.138.0.3-5, confirming they were on the same /24 subnet as my scanning host.

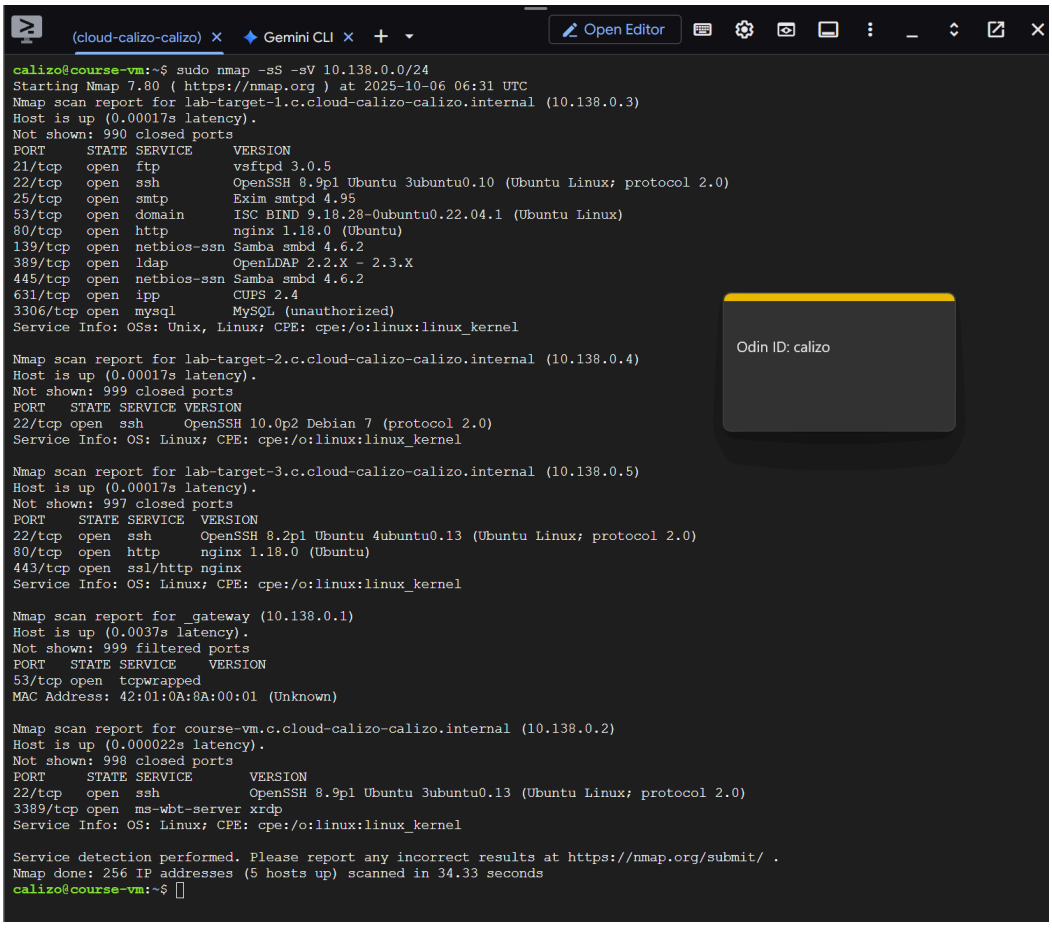
Step 3 - Scanning the Subnet for Active Services

After reconnecting to my course-vm, I performed an internal network sweep:

- sudo nmap -sS -sV 10.138.0.0/24

This SYN scan with version detection revealed active services and open ports across the subnet. The resulting output (shown in the attached screenshot) identified multiple running services on each of the three targets, including SSH, HTTP, FTP, and MySQL, depending on the image configuration.

The screenshot below captures the complete nmap output verifying that five hosts were detected on the network - the three targets, the gateway, and my scanning VM (also including host discovery, open ports, and service versions).



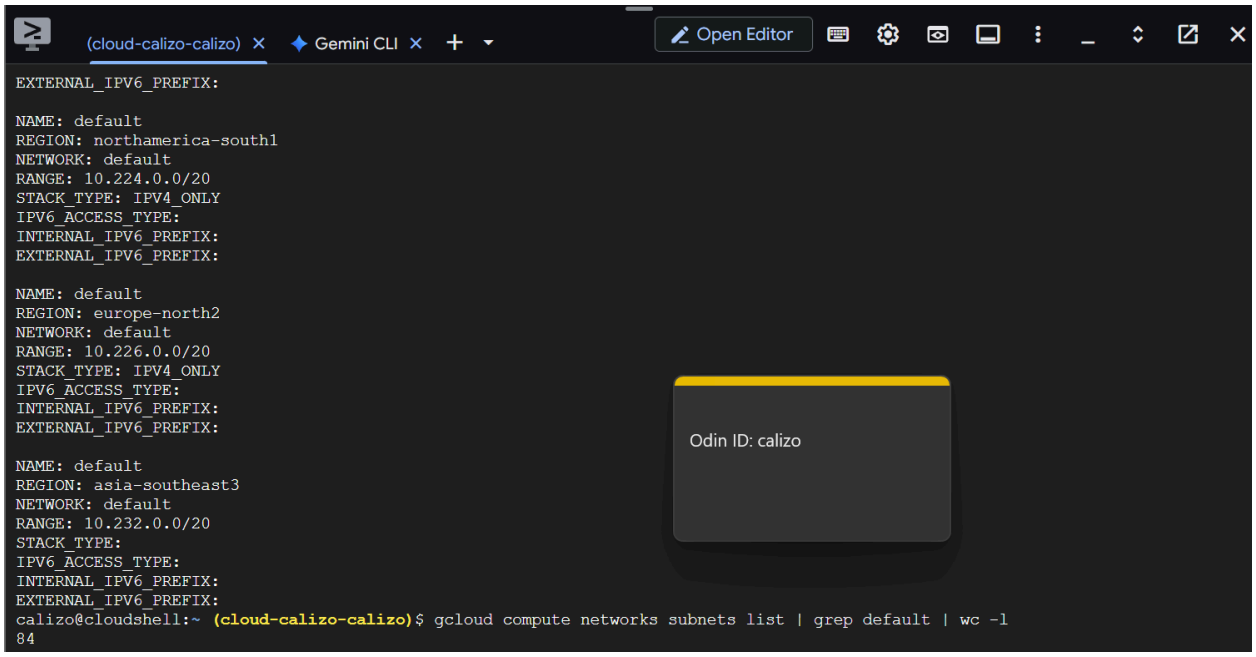
Once the scan was verified and documented, I shut down all three target VMs.

Insight

Running nmap on an internal subnet felt like watching the invisible become visible - each port a small heartbeat of purpose. The scan turned raw IP addresses into living systems, each with its own exposed surface and hidden logic. Understanding what is open, and why, is the foundation of responsible network defense.

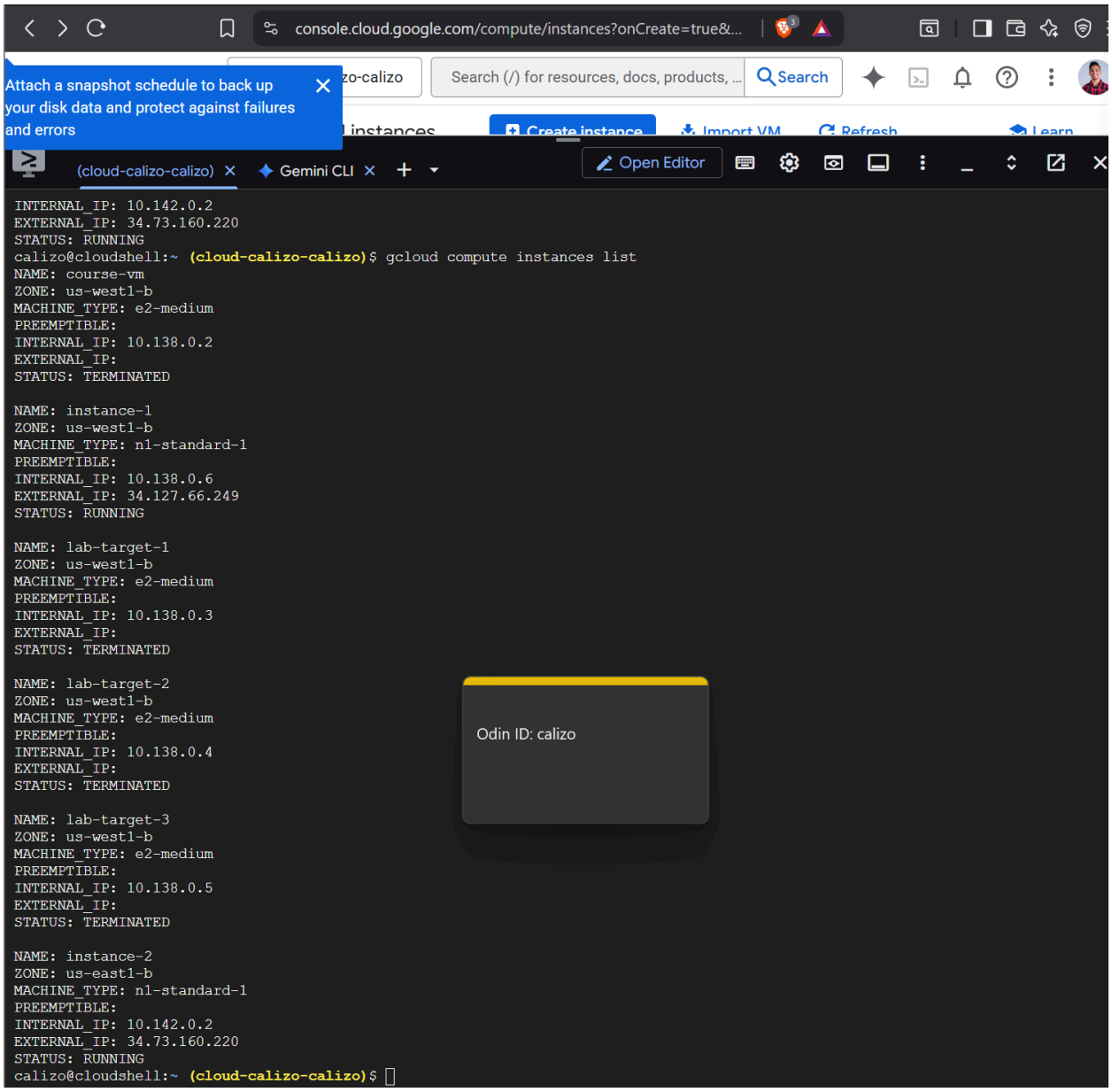
Step 4-5: CIDR and Default Network Exploration

After reviewing how legacy subnets can be lifted and shifted into the cloud, I explored the structure of Google Cloud's default network using the gcloud CLI. By running gcloud compute networks subnets list | grep default | wc -l, I confirmed that **84 default subnetworks** are pre-configured across all available regions. Each region receives its own /20 CIDR-based subnetwork, giving roughly **4,096 usable host addresses per region** - a comfortable range for scalable deployment within each availability zone.



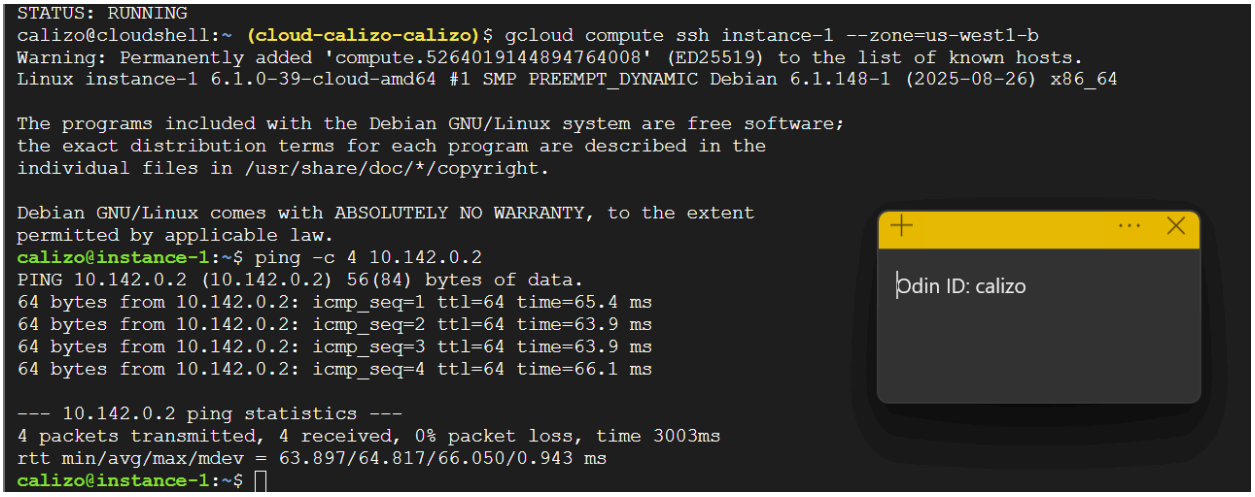
To observe this in practice, I spun up two compute instances in separate regions using:

- gcloud compute instances create instance-1 --zone=us-west1-b
- gcloud compute instances create instance-2 --zone=us-east1-b



Each instance automatically mapped to its region's respective default subnet - **10.138.0.0/20 for us-west1-b** and **10.142.0.0/20 for us-east1-b**. This mapping confirmed that Google Cloud intelligently assigns CIDR subnetworks based on region during instance creation.

Next, I SSHed into instance-1 and used ping to reach the internal IP of instance-2 (10.142.0.2). The ping succeeded with consistent low-latency responses:



This validated how **GCP's virtual switch layer** - not the VPN gateway - facilitates seamless internal communication across zones within the same project. The underlying SDN (software-defined networking) abstracts the inter-region transport, allowing what feels like a flat network space even though the machines are deployed continents apart.

Step 6: Creating Custom Networks

In this step, I explored Google Cloud's flexibility in designing custom virtual networks beyond its default VPC configuration. Using Cloud Shell, I created a new network named custom-network1 with subnet-mode set to custom, allowing explicit control over subnet configurations.

Two subnetworks were deployed within this custom network - one in us-central1 (192.168.1.0/24) and another in europe-west1 (192.168.5.0/24). The screenshots confirm successful creation and verification using gcloud compute networks subnets list and region-specific listing commands. This setup demonstrates how organizations can segment infrastructure geographically while maintaining IP management flexibility.

```
calizo@cloudshell:~ (cloud-calizo-calizo)$ gcloud compute networks subnets create subnet-europe-west-192 \
  --network custom-network1 \
  --region europe-west1 \
  --range 192.168.5.0/24
Created [https://www.googleapis.com/compute/v1/projects/cloud-calizo-calizo/regions/europe-west1/subnetworks/subnet-europe-west-192].
NAME: subnet-europe-west-192
REGION: europe-west1
NETWORK: custom-network1
RANGE: 192.168.5.0/24
STACK_TYPE: IPV4_ONLY
IPV6_ACCESS_TYPE:
INTERNAL_IPV6_PREFIX:
EXTERNAL_IPV6_PREFIX:
calizo@cloudshell:~ (cloud-calizo-calizo)$ gcloud compute networks subnets list --network custom-network1
NAME: subnet-us-central-192
REGION: us-central1
NETWORK: custom-network1
RANGE: 192.168.1.0/24
STACK_TYPE: IPV4_ONLY
IPV6_ACCESS_TYPE:
INTERNAL_IPV6_PREFIX:
EXTERNAL_IPV6_PREFIX:

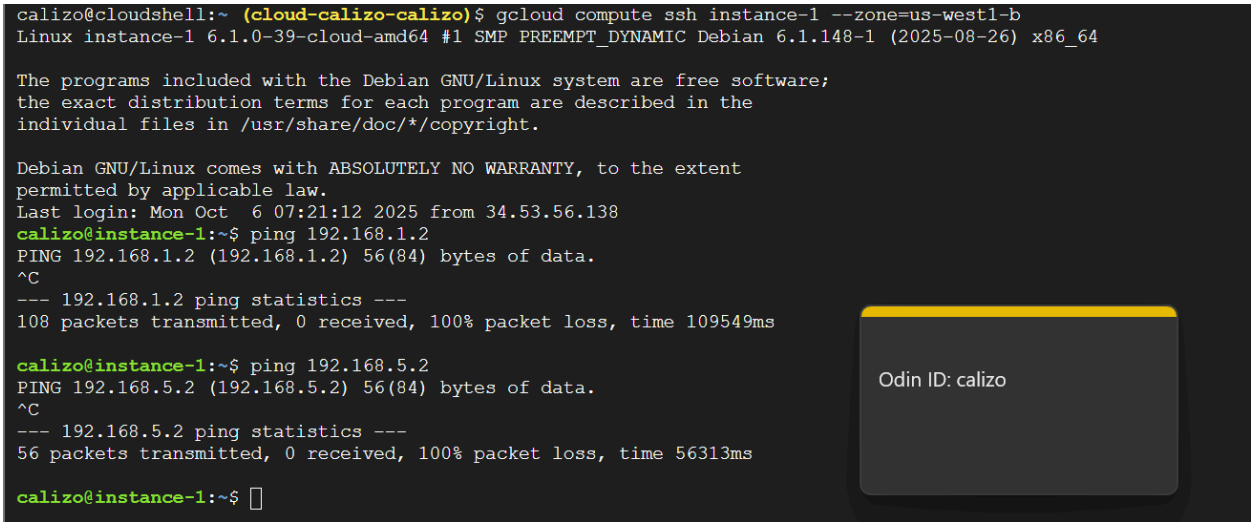
NAME: subnet-europe-west-192
REGION: europe-west1
NETWORK: custom-network1
RANGE: 192.168.5.0/24
STACK_TYPE: IPV4_ONLY
IPV6_ACCESS_TYPE:
INTERNAL_IPV6_PREFIX:
EXTERNAL_IPV6_PREFIX:
calizo@cloudshell:~ (cloud-calizo-calizo)$ gcloud compute networks subnets list --regions=us-central1,europe-west1
NAME: default
REGION: europe-west1
NETWORK: default
RANGE: 10.132.0.0/20
STACK_TYPE: IPV4_ONLY
IPV6_ACCESS_TYPE:
INTERNAL_IPV6_PREFIX:
EXTERNAL_IPV6_PREFIX:

NAME: subnet-europe-west-192
REGION: europe-west1
NETWORK: custom-network1
RANGE: 192.168.5.0/24
STACK_TYPE: IPV4_ONLY
IPV6_ACCESS_TYPE:
INTERNAL_IPV6_PREFIX:
EXTERNAL_IPV6_PREFIX:

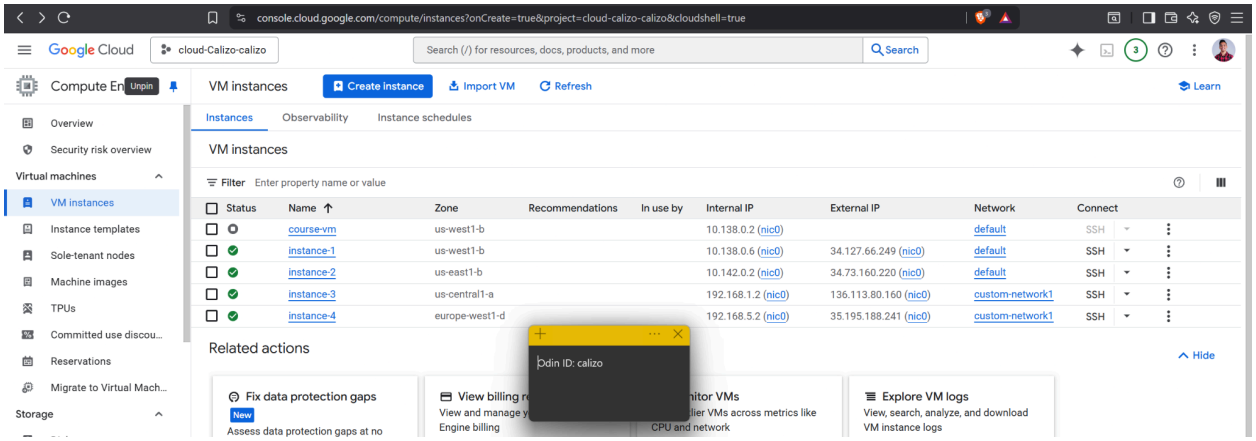
NAME: default
REGION: us-central1
NETWORK: default
RANGE: 10.128.0.0/20
STACK_TYPE: IPV4_ONLY
IPV6_ACCESS_TYPE:
INTERNAL_IPV6_PREFIX:
EXTERNAL_IPV6_PREFIX:

NAME: subnet-us-central-192
REGION: us-central1
NETWORK: custom-network1
RANGE: 192.168.1.0/24
STACK_TYPE: IPV4_ONLY
IPV6_ACCESS_TYPE:
INTERNAL_IPV6_PREFIX:
EXTERNAL_IPV6_PREFIX:
calizo@cloudshell:~ (cloud-calizo-calizo)$
```

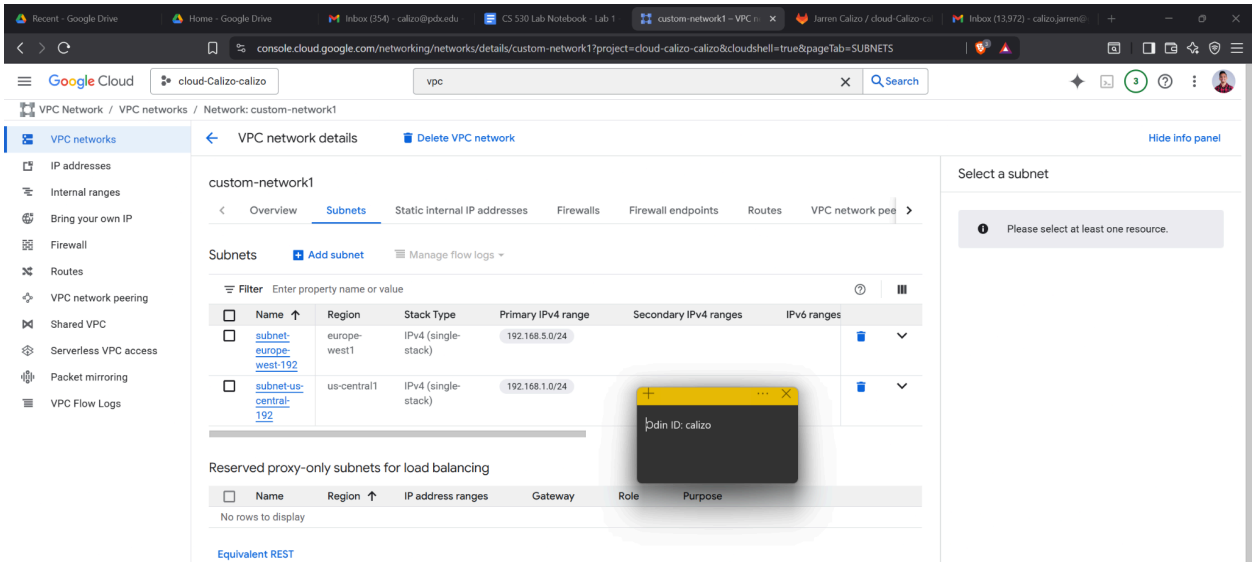

Next, I instantiated instance-3 and instance-4 in each custom subnetwork, complementing the default network’s instance-1 and instance-2 from previous steps. When I attempted to ping from instance-1 (default network) to the internal IPs of instance-3 and instance-4, the results showed 100% packet loss.

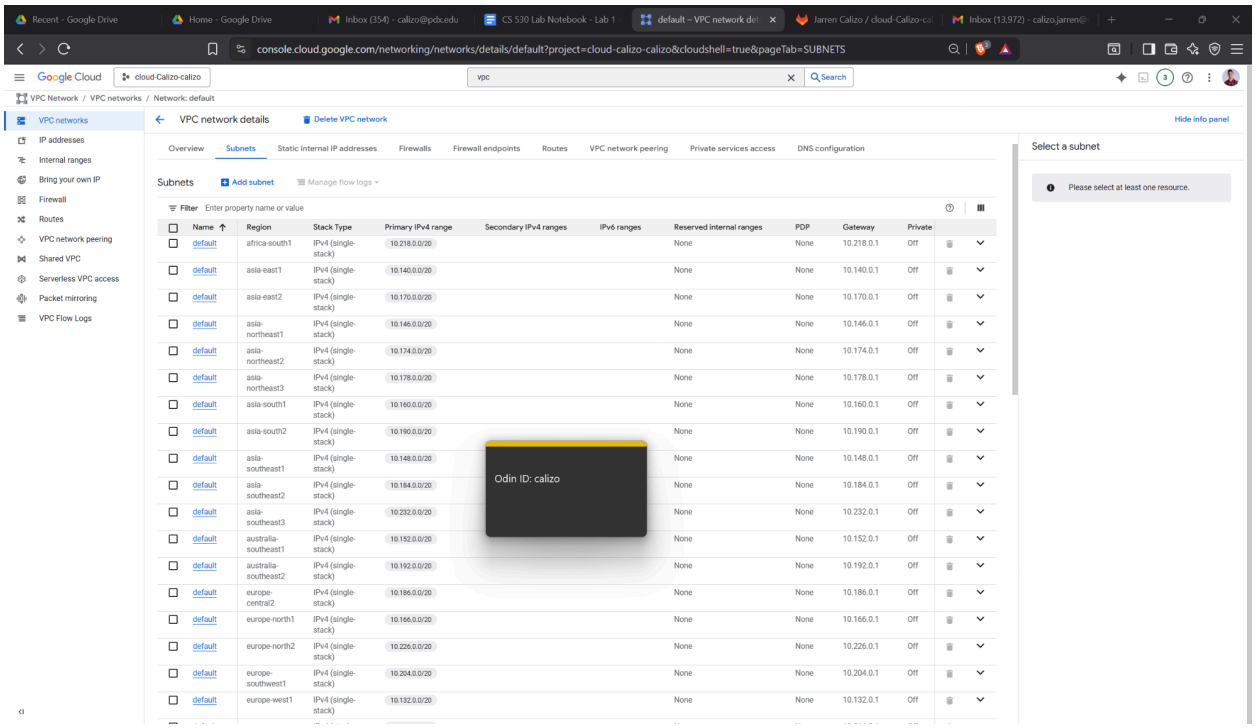


The ping failed because these instances reside in different VPC networks. By design, GCP isolates traffic between networks unless VPC peering or Shared VPC connections are explicitly configured. In this case, while instances in the same default network could communicate through an implicit virtual switch, the custom-network1 instances remained unreachable, protected by GCP’s internal segmentation of network namespaces.



To visualize the setup, the Compute Engine UI screenshot shows all four instances (instance-1 through instance-4) across both networks with their internal and external IPs clearly labeled. Likewise, the VPC Network pages confirm the two custom subnetworks (subnet-us-central-192 and subnet-europe-west-192) under custom-network1 alongside the numerous global default subnets automatically provisioned by GCP.





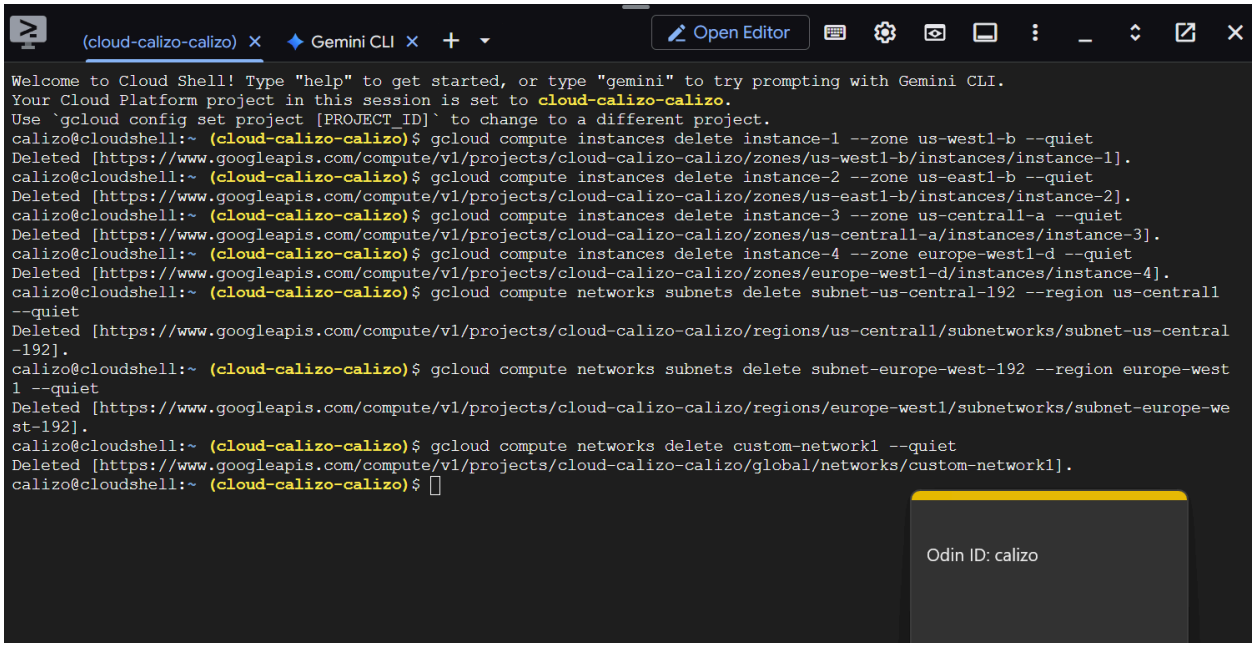
Step 7: Clean Up

After validating all configurations, I followed best practice by cleaning up all resources to avoid unnecessary compute charges. Using the --quiet flag, I deleted:

- All four VM instances across the us-west1-b, us-east1-b, us-central1-a, and europe-west1-d zones.
- Both custom subnetworks created for custom-network1.

- Finally, the custom-network1 network itself.

This orderly teardown is shown in the final screenshot, verifying successful deletions for each resource.



Final Insight

These final steps were a fitting conclusion to the lab, encapsulating the balance between automation and control that defines modern cloud networking. By witnessing how subnet boundaries dictate connectivity - and how segmentation strengthens isolation - I deepened my understanding of how GCP enforces security through architectural separation rather than firewalls alone. The process felt like assembling and then gracefully dismantling a cloud-scale topology, reinforcing the mindset that every network design, no matter how transient, should be both intentional and secure.